WatchTower: Observation-First Forensics for Multi-Agent AI Systems

Detecting Silent Failures and Report-vs-Reality Gaps When an Agent's Self-Report Is Not Ground Truth

Rohit Jinsiwale
Proof-of-Concept Artifact
https://github.com/beejak/agentwatch

***Abstract*—Monitoring tools for LLM-agent systems consume what an agent *reports* it did—the trace of tool calls and outputs it emits. We argue this is insufficient as a security signal: agents fail in ways their self-report does not reveal, and a compromised or buggy agent (or tool) can under-report its own actions. We characterize two structural blind spots—*silent failures* (the agent reports success while looping, burning tokens, or collapsing in output diversity) and *cross-layer discrepancies* (the agent's reported actions disagree with an independent host view)—and present WATCHTOWER, an observation-first platform that detects them by reconciling the self-report against (a) the *shape* of the execution trajectory and (b) an independent *host* ground truth. On a frozen, held-out corpus a faithful self-report baseline detects 0% of both classes *by construction*, while WATCHTOWER detects silent failures at 0.86 recall (0.07 FP) and cross-layer discrepancies at 1.00 (0.00 FP). We confirm the mechanism on real, emergent traffic from an LLM agent whose behavior is captured through an egress proxy (n=120; recall 1.00, FP 0.00), and we map the detection envelope—including where detection falls off. This paper documents a proof-of-concept artifact released for reproduction and extension; WATCHTOWER observes and attributes but does not enforce (a co-designed enforcement layer is described separately).**

I. Introduction

LLM agents increasingly act through tools, memory, and peer agents. Existing observability (LangSmith, Langfuse) records the agent's emitted trace and renders it on a dashboard. That trace is the agent's *self-report*, and a self-report is not ground truth:

- **The silent failure.** An agent retries the same call 150 times; every span is `status=ok`; cost balloons. Nothing *errored*—an output-level monitor shows a green dashboard while the task was broken throughout.
- **The lying agent.** An agent's trace reports one network call; the host observed four. The three unreported calls are invisible to anything that trusts the trace—an exfiltration/tampering signal.

We argue monitoring must *reconcile the self-report against independent evidence*—the trajectory's shape and an independent host view—and that the discrepancy is the signal.

Contributions.

- **C1, silent-failure detection (SC2):** cost-anomaly, retry-loop, and entropy-collapse detectors that flag failures which never raise an error.
- **C2, cross-layer reconciliation (SC3):** comparison of agent-reported network actions against host telemetry, scored as a tamper/exfil discrepancy.
- **C3, an append-only forensic Chronicle + attribution (SC1):** a no-UPDATE/no-DELETE event store enabling post-hoc attribution of a coordination failure to agent, action, and call-tree position.
- **Evidence:** a frozen, held-out evaluation with baselines and confidence intervals, an operating-envelope sweep, and confirmation on real emergent LLM-agent traffic. All artifacts are released for reproduction.

II. Background and Related Work

Trace/output monitors (LangSmith, Langfuse) record and visualize the emitted trace; they trust the self-report and thus cannot flag an all-`ok` silent failure, nor detect a report-vs-reality gap (no host concept). **The observability gap** [1] argues output-level feedback is insufficient for coding agents; we operationalize that argument with concrete detectors and an evaluation. **Agentic process observability** [2] treats trajectories as process-mining targets and highlights stochasticity; it is complementary, not a security discrepancy detector. **Host EDR** (Sysmon, Falco, eBPF) observes OS-level network/process events but has no agent semantics; WATCHTOWER *bridges* host events to the agent trajectory via process GUIDs. The **MAST** taxonomy [3] informs the coordination-failure signatures used in attribution. Our novelty is *multi-layer reconciliation (self-report ↔ trajectory ↔ host) as a security signal*, with concrete detectors, a forensic substrate, and a held-out evaluation.

III. Threat and Failure Model

We consider agents that may report success while failing—non-adversarial bugs (retry storms, token burn) and adversarial masking—and a compromised/injected agent (or tool) that under-reports network activity (SC3 exfil). *In scope:* detection, attribution, forensic record. *Out of scope (companion enforcement system):* blocking, prompt-injection content classification, and cross-session taint.

IV. System Design

WATCHTOWER is a passive, append-only, 16-layer side-car (Fig. 1). Agents emit HMAC-signed `Signals` to a Redis stream; a Receiver verifies origin on every signal and fans

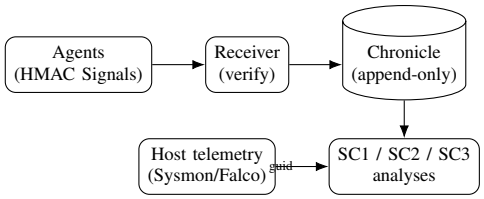

Fig. 1. Observe-and-reconcile pipeline. The Chronicle is the forensic source of truth; SC3 reconciles the agent self-report against host telemetry by process GUID.

out to inspection layers; everything is recorded to an append-only **Chronicle** (ClickHouse; no UPDATE/DELETE; 90-day TTL). Analyses run over the Chronicle.

SC1 attribution. From a trace's spans, identify the failing agent/action and call-tree position, classified against a coordination-signature (MAST) library.

SC2 silent failure. Over a trace's spans: a cost-anomaly ratio (actual/expected), a repeated-summary retry-loop test, and an entropy-collapse test—none of which require an error status.

SC3 cross-layer. Count agent-reported network actions; count host `NetworkConnect` events for the same process GUIDs; a positive delta (host $>$ reported) is the discrepancy.

A three-stage verdict engine (deterministic $\rightarrow$ behavioral-baseline $\rightarrow$ sampled LLM judge) and a per-agent behavioral baseline (3σ; restricted until 50 traces) support the analyses.

V. Implementation

Python 3.12 (all-async), Pydantic v2 (the `Signal` shape defined once), a FastAPI surface, a ClickHouse Chronicle, Redis transport, PostgreSQL baseline, Neo4j access graph, HMAC-SHA256 signing, and 12-factor environment configuration. A lightweight SDK lets any agent emit signed signals to the Chronicle or the `wt:signals` stream. Approximately 16 layers, each gated by a test. The LLM judge is a real OpenAI-compatible client when configured, and a deterministic heuristic otherwise—so the pipeline and CI run with zero LLM dependency.

VI. Evaluation

We use frozen, human-labeled corpora; a held-out split; baselines head-to-head; and bootstrap 95% confidence intervals. Baselines: **B1** self-report (flags only an explicit error span—the faithful upper bound of any self-report-trusting monitor) and **B2** a naive cost threshold. All numbers are reproducible from the released artifacts: the synthetic corpus is `eval/corpus/traces_v0.1.jsonl` (results `eval/results/traces_v0.1_test.json`); the real-LLM corpus `eval/corpus/captured_llm_v0.1.jsonl` (`eval/results/captured_llm_test.json`); the envelope sweep `eval/results/sweep_v0.1.json`.

H3 (the spine). The self-report baseline has **0 recall** on both classes—it never sees the signal (Fig. 2). WatchTower also beats naive cost (loops/entropy catch what pure cost misses; pure cost false-positives on expensive-but-correct work).

TABLE I
Held-out synthetic corpus (n=269; 380-trace corpus).

Detector	SC2 recall	SC2 FPR	SC3 recall	SC3 FPR
WatchTower	**0.86**	0.07	**1.00**	0.00
Self-report (B1)	0.00	0.00	0.00	0.00
Naive cost (B2)	0.27	0.07	—	—

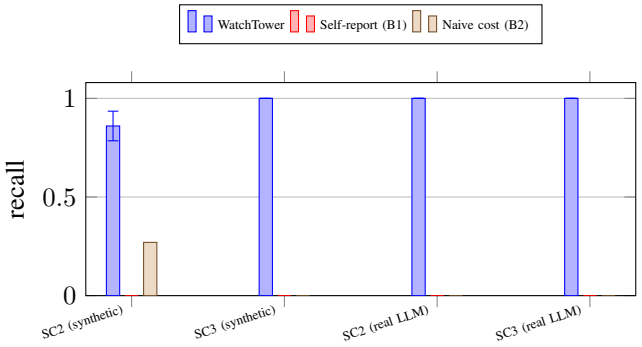

Fig. 2. Detection recall by detector, on held-out synthetic and real LLM-driven traffic. **H3:** the self-report baseline is structurally blind (0.00) across the board, while WatchTower detects.

Near-real-world methodology. A DeepSeek agent (ReAct loop) acts against tasks; its tools make real HTTP egress through an independent egress proxy (mitmproxy) that records every call regardless of whether the agent emitted a span. SC2 (the model genuinely loops) and SC3 (a compromised tool's hidden egress) *emerge* from the model's behavior—nothing is scripted. Table III confirms H3 on real traffic. The 1.00 reflects a clean signal once the condition occurs; the conservative headline is the synthetic 0.86 of Table I, which includes borderline cases.

A. Comparison to existing monitoring

Trace/output monitors trust the self-report (no host view, no silent-failure model); host EDR sees the host but not agent intent. Only WatchTower reconciles the two (Table IV).

B. Practicality

Detection is cheap and runs off the agent's hot path (Table **??**, commodity CPU, no GPU/network): all detectors are sub-millisecond per trace, and a full SC1+SC2+SC3 pass sustains ~66k spans/s.

C. Case study: what the dashboard hides

VII. Discussion and Limitations

- **Attribution is first-error, not causal-root.** On cascade failures where the root error is not the first error span, SC1 mis-identifies the agent (Table II). Causal-root attribution is future work.
- **Synthetic realism.** The bulk corpus is generated; the robust result is the structural *baseline gap*, confirmed on real LLM-agent traffic. Absolute numbers will move on production traffic.

TABLE II
OPERATING ENVELOPE—WHERE DETECTION HOLDS AND FALLS OFF.

Condition	Boundary
SC2 silent loop	fires at loop length ≥ 10 (shorter missed)
SC2 token burn	fires at total cost $\geq \$0.12$ (FP above on costly-OK)
SC3 cross-layer	fires at delta ≥ 1; negative delta never fires
SC1 attribution	0.50: cascade exposes first-error $\neq$ causal-root

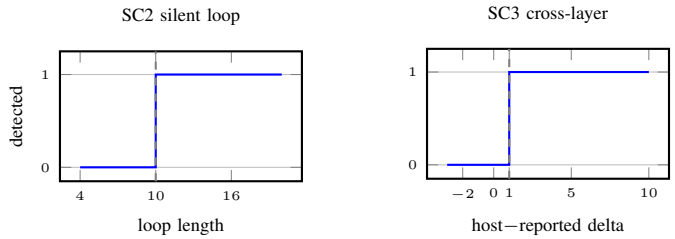

Fig. 3. Operating envelopes. SC2 silent-loop detection turns on at loop length ≥ 10 (shorter loops are honest misses); SC3 fires at host−reported delta ≥ 1 and never on negative delta (telemetry sampling). Token-burn fires at cost $\geq \$0.12$.

- **SC3 without kernel telemetry** validates HTTP egress only (proxy); raw-socket, DNS, and proxy-bypass require an eBPF/Sysmon collector (a bridge is implemented and unit-tested).
- **Captured scale.** The real-agent corpus is 120 traces—ample to confirm the mechanism, modest for headline claims; the harness scales with a parameter.
- **LLM judge is fallible**—it is the last, sampled verdict stage by design; deterministic stages are authoritative.

VIII. CONCLUSION

Treating an agent's self-report as untrusted—and reconciling it against the trajectory and an independent host view—catches silent failures and report-vs-reality gaps that self-report-based monitoring structurally cannot. We release the implementation, frozen corpora, and harness as a proof-of-concept artifact for reproduction and extension.

ARTIFACT AVAILABILITY

All code, frozen corpora, results, and this paper are released at https://github.com/beejak/agentwatch. Reproduce: `make eval` (held-out synthetic, Table I), `python -m eval.sweep` (operating envelope, Table II and Fig. 3), and `make capture-tier1-llm` (real LLM-driven capture, Table III). Data: `eval/corpus/traces_v0.1.jsonl` (n=380) and `eval/corpus/captured_llm_v0.1.jsonl` (n=120); per-run metrics in `eval/results/*.json`; end-to-end demo output in `examples/sample_output.json`; methodology in `docs/TESTING.md` and `docs/REAL_TRAFFIC_VALIDATION.md`. The companion enforcement system is at https://github.com/beejak/agentwatch-firewall.

TABLE III
NEAR-REAL-WORLD: REAL LLM AGENT, EMERGENT TRAFFIC (n=120, 40/CLASS).

Detector	SC2 recall	SC2 FPR	SC3 recall	SC3 FPR
WatchTower	**1.00**	0.00	**1.00**	0.00
Self-report (B1)	0.00	0.00	0.00	0.00

TABLE IV
CAPABILITY COMPARISON (✓ DETECTS/PROVIDES; × CANNOT).

Capability	WatchTower	LangSmith/Langfuse	Host EDR
SC1 attribution (call-tree)	✓	partial	×
SC2 silent failure	✓	×	×
SC3 cross-layer discrepancy	✓	×	×†
Append-only forensic audit	✓	×	partial
Agent semantics	✓	✓	×

†Host EDR sees host events but has no agent self-report to reconcile against.

REFERENCES

[1] "The Observability Gap in LLM Coding Agents," arXiv:2603.26942, 2026.
[2] "Agentic AI Process Observability," arXiv:2505.20127, 2025.
[3] Cohen et al., "MAST: A Multi-Agent System Taxonomy for LLM Failure Mode Classification," NeurIPS, 2025.
[4] OWASP, "Top 10 for LLM Applications," 2025.
[5] Microsoft, "Sysmon," Sysinternals; Falco, "Cloud-native runtime security."